Senior CMS Developer – Task

Dynamic Custom Product Add-on Block & Custom Bundle Builder Section

Dear Candidate,

*Thank you for your interest in the **Senior CMS Developer** position at **Ropstam Solutions**. As part of our hiring process, we would like you to complete the following technical task to demonstrate your hands-on expertise in WordPress and Shopify development, code quality, platform best practices, and ability to build practical, production-ready solutions.*

This exercise evaluates your hands-on skills in WordPress and Shopify development, including PHP, JavaScript, Liquid, APIs, dynamic functionality, and your approach to performance, security, accessibility, and edge cases.

For any queries regarding the task, please reach us at jobs@ropstam.com.

- **Time Allocated:** 48 Hours from receipt of this brief
- **Submission Format:** GitHub Repository + Required Preview/Demo Links

Objective

Build a dual-part technical test to evaluate code quality, platform convention compliance, and real-world execution.

WordPress Task – Dynamic Custom Product Add-on Block

- **Scenario**

  Build a custom WordPress block / lightweight custom plugin.
- **Requirements**
  a. Register a custom post type Services with two custom meta fields (`base_price` and `delivery_time_days`).
  b. Create a block (or shortcode/template component) that fetches the 3 most recently updated services.
  c. Include a client-side calculator (vanilla JS) that lets users toggle optional add-ons (e.g., "Priority Turnaround: +$50") and updates the estimated price live on screen.
  d. Ensure all database operations use prepared queries or `WP_Query`, output is sanitized/escaped (`esc_html`, `esc_attr`), and assets are registered strictly via `wp_enqueue_script` with dependencies specified.

Shopify Task – Custom Bundle Builder Section

- **Scenario**

  Build an Online Store 2.0 native Section using Shopify CLI.
- **Requirements**
  a. Build a custom Liquid section (`bundle-builder.liquid`) containing schema settings allowing merchants to select up to 3 specific collections or product lists.
  b. Implement an interactive UI where customers select 1 item from each group to create a 3-pack bundle.
  c. Dynamically update the bundle price and use the Shopify Ajax API (`/cart/add.js`) to add all 3 items to the cart simultaneously using line item properties linking them as a bundle.
  d. Display proper stock/sold-out validation and loading states without reloading the page. Clean, unbloated Liquid and vanilla JavaScript only (no heavy libraries).

Evaluation Rubric (100 Points Total)

Category	Weight	Evaluation Criteria
Platform Standards	30%	Proper Liquid filters, schema definitions, WordPress hook choices, and naming conventions.
Performance & Code	25%	Minimal DOM manipulation, zero unneeded dependencies, modular CSS/JS, no raw queries.
State & Edge Cases	20%	Handling out-of-stock items, API failure fallbacks, mobile responsiveness, and sanitization.
Git & Documentation	15%	Atomic commits, clear setup instructions in a README.md, and clean code comments.
Accessibility	10%	Semantic markup, keyboard navigation on buttons/toggles, and proper ARIA states.

Submission Guidelines

Your submission must be shared within the agreed timeframe and contain the following:

- **GitHub Repository:** One repository containing both the WordPress and Shopify implementations, along with a clear README.md covering setup instructions.
- **WordPress Demo Link:** A working demo/preview of your completed WordPress implementation.
- **Shopify Demo Link:** A working demo/development store link showing your completed Shopify implementation, along with the storefront password if required.
- **Documentation:** Briefly mention any required configurations, dependencies, assumptions, or limitations.